

Arhitectura Calculatoarelor

Proiect: Implementarea Procesorului MIPS 32
Arhitecturi Ciclu Unic și Pipeline

Problemă: Suma puterilor lui 2 dintr-un interval $[a, b]$

Student:

Măgerușan Alexandru

Data:

23 mai 2026

Cuprins

1	Introducere	2
2	Algoritmul Soluției	2
2.1	Cod Sursă C++	2
2.2	Cod Assembly x86	3
2.3	Cod Assembly MIPS	4
3	Cod Mașină și Maparea ROM	5
I	Implementarea cu Ciclu Unic	6
4	Setul de Instrucțiuni și Semnalele de Control	6
4.1	Clasificarea Instrucțiunilor și Calea de Date	6
4.1.1	Instrucțiuni de Tip R (Register)	6
4.1.2	Instrucțiuni de Tip I (Immediate)	6
4.1.3	Instrucțiuni de Tip J (Jump)	8
5	Componentele Hardware și Arhitectura	8
5.1	Diagrama Bloc a Procesorului	8
5.2	IFetch.vhd	9
5.3	IDecode.vhd	10
5.4	EX.vhd	11
5.5	MEM.vhd	12
5.6	UC.vhd	13
5.7	test_env.vhd	14
6	Urmărirea Execuției în Ciclu Unic	17
II	Implementarea Pipeline	18
7	Registreele Pipeline și Arhitectura	18
8	Semnalele de Control în Pipeline	19
9	Rezolvarea Hazardurilor	19
10	Urmărirea Execuției în Pipeline	20
10.1	Execuție Ideală (Identificarea Hazardurilor)	20
11	Modificările Codului VHDL (Pipeline)	23
III	Concluzii	26
12	Sinteza FPGA	26

1 Introducere

Acest proiect prezintă proiectarea hardware a unui microprocesor MIPS pe 32 de biți implementat în VHDL. Implementarea este împărțită în două etape evolutive distincte:

- **Faza 1:** O arhitectură fundamentală cu ciclu unic.
- **Faza 2:** O arhitectură avansată în 5 etape (pipeline) cu rezolvarea hazardurilor.

Sistemul este validat prin rezolvarea unei probleme specifice: calcularea sumei puterilor lui 2 dintr-un set de date dat, restricționat condiționat la un interval specificat.

2 Algoritmul Soluției

2.1 Cod Sursă C++

Implementarea de referință de nivel înalt a algoritmului.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int a = 2;
6      int b = 90;
7      int x[] = {0, 1, 2, 3, 256, 14, 16};
8      int n = 7;
9
10     int sum = 0;
11     int i = -1;
12
13     while (i < n - 1)
14     {
15         i++;
16         int val = x[i];
17
18         if (val < a) continue;
19         if (val > b) continue;
20         if (val <= 0) continue;
21
22         int temp = val - 1;
23         int test = val & temp;
24
25         if (test != 0) continue;
26
27         sum = sum + val;
28     }
29
30     printf("Suma:%d\n", sum);
31     return 0;
32 }
```

Program de Referință C++

2.2 Cod Assembly x86

Cod x86 generat pentru testarea logicii de procesare.

```
1 main:
2     push    rbp
3     mov     rbp, rsp
4     mov     DWORD PTR [rbp-12], 2
5     mov     DWORD PTR [rbp-16], 90
6     mov     DWORD PTR [rbp-64], 0
7     mov     DWORD PTR [rbp-60], 1
8     mov     DWORD PTR [rbp-56], 2
9     mov     DWORD PTR [rbp-52], 3
10    mov     DWORD PTR [rbp-48], 256
11    mov     DWORD PTR [rbp-44], 14
12    mov     DWORD PTR [rbp-40], 16
13    mov     DWORD PTR [rbp-20], 7
14    mov     DWORD PTR [rbp-4], 0
15    mov     DWORD PTR [rbp-8], -1
16    jmp     .L2
17 .L7:
18     add     DWORD PTR [rbp-8], 1
19     mov     eax, DWORD PTR [rbp-8]
20     cdqe
21     mov     eax, DWORD PTR [rbp-64+rax*4]
22     mov     DWORD PTR [rbp-24], eax
23     mov     eax, DWORD PTR [rbp-24]
24     cmp     eax, DWORD PTR [rbp-12]
25     jge     .L3
26     jmp     .L2
27 .L3:
28     mov     eax, DWORD PTR [rbp-24]
29     cmp     eax, DWORD PTR [rbp-16]
30     jle     .L4
31     jmp     .L2
32 .L4:
33     cmp     DWORD PTR [rbp-24], 0
34     jg      .L5
35     jmp     .L2
36 .L5:
37     mov     eax, DWORD PTR [rbp-24]
38     sub     eax, 1
39     mov     DWORD PTR [rbp-28], eax
40     mov     eax, DWORD PTR [rbp-24]
41     and     eax, DWORD PTR [rbp-28]
42     mov     DWORD PTR [rbp-32], eax
43     cmp     DWORD PTR [rbp-32], 0
44     je      .L6
45     jmp     .L2
46 .L6:
47     mov     eax, DWORD PTR [rbp-24]
48     add     DWORD PTR [rbp-4], eax
49 .L2:
50     mov     eax, DWORD PTR [rbp-20]
51     sub     eax, 1
52     cmp     DWORD PTR [rbp-8], eax
53     jl      .L7
54     mov     eax, 0
55     pop     rbp
56     ret
```

Cod Assembly x86

2.3 Cod Assembly MIPS

Implementarea finală destinată execuției pe procesorul MIPS.

```
1  .data
2      # Declararea array-ului n memorie (fiecare num r ocup 1 word = 4
      #   octe i)
3      x: .word 0, 1, 2, 3, 256, 14, 16
4
5  .text
6  .globl main
7  main:
8      addi $1, $0, 2      # a = 2
9      addi $2, $0, 90     # b = 90
10     addi $3, $0, 7      # n = 7
11     la    $4, x          # base_addr = adresa de nceput a array-ului x
12     add   $5, $0, $0     # sum = 0
13     add   $6, $0, $0     # i = 0
14
15 loop_start:
16     slt   $7, $6, $3     # $7 = 1 dac i < n
17     beq   $7, $0, end    # Dac i >= n, terminat, sare la 'end'
18
19     # --- COREC IA ADRESEI ---
20     sll   $9, $6, 2      # $9 = i * 4
21     add   $9, $4, $9     # $9 = adresa elementului curent
22     lw    $8, 0($9)     # val = cite te x[i] n $8
23
24     addi  $6, $6, 1      # i = i + 1
25
26     # Verificare: val < a
27     slt   $7, $8, $1     # $7 = 1 dac val < a
28     bne   $7, $0, loop_start
29
30     # Verificare: val > b
31     slt   $7, $2, $8     # $7 = 1 dac b < val
32     bne   $7, $0, loop_start
33
34     # Verificare: val > 0
35     slt   $7, $0, $8     # $7 = 1 dac 0 < val
36     beq   $7, $0, loop_start
37
38     # Verificare: este putere a lui 2? (val & (val - 1) == 0)
39     addi  $9, $8, -1     # $9 = val - 1
40     and   $9, $8, $9     # $9 = val & (val - 1)
41     bne   $9, $0, loop_start
42
43     # Adaug la sum
44     add   $5, $5, $8     # sum = sum + val
45     j     loop_start
46
47 end:
48     addi  $v0, $0, 10    # ie ire
49     syscall
```

Cod Assembly MIPS pentru procesor

3 Cod Mașină și Maparea ROM

Următorul tabel detaliază traducerea binară și hexazecimală a codului Assembly MIPS, așa cum este introdus în memoria de instrucțiuni (ROM) pentru ambele arhitecturi.

PC	Cod Mașină (Binar)	Hex	MIPS ASM
0	001000_00000_00001_00000000000000010	20010002	addi \$1, \$0, 2
1	001000_00000_00010_0000000001011010	2002005A	addi \$2, \$0, 90
2	001000_00000_00011_00000000000000111	20030007	addi \$3, \$0, 7
3	000000_00000_00000_00100_00000_100000	00002020	add \$4, \$0, \$0 (<i>la \$4, x</i>)
4	000000_00000_00000_00101_00000_100000	00002820	add \$5, \$0, \$0
5	000000_00000_00000_00110_00000_100000	00003020	add \$6, \$0, \$0
— <i>loop_start: (PC = 6)</i> —			
6	000000_00110_00011_00111_00000_101010	00C3382A	slt \$7, \$6, \$3
7	000100_00111_00000_00000000000001111	10E0000F	beq \$7, \$0, end (<i>+15</i>)
8	000000_00000_00110_01001_00010_000000	00064880	sll \$9, \$6, 2
9	000000_00100_01001_01001_00000_100000	00894820	add \$9, \$4, \$9
10	100011_01001_01000_00000000000000000	8D280000	lw \$8, 0(\$9)
11	001000_00110_00110_00000000000000001	20C60001	addi \$6, \$6, 1
12	000000_01000_00001_00111_00000_101010	0101382A	slt \$7, \$8, \$1
13	000101_00111_00000_1111111111111000	14E0FFF8	bne \$7, \$0, loop_start (<i>-8</i>)
14	000000_00010_01000_00111_00000_101010	0048382A	slt \$7, \$2, \$8
15	000101_00111_00000_1111111111110110	14E0FFF6	bne \$7, \$0, loop_start (<i>-10</i>)
16	000000_00000_01000_00111_00000_101010	0008382A	slt \$7, \$0, \$8
17	000100_00111_00000_1111111111110100	10E0FFF4	beq \$7, \$0, loop_start (<i>-12</i>)
18	001000_01000_01001_1111111111111111	2109FFFF	addi \$9, \$8, -1
19	000000_01000_01001_01001_00000_100100	01094824	and \$9, \$8, \$9
20	000101_01001_00000_1111111111110001	1520FFF1	bne \$9, \$0, loop_start (<i>-15</i>)
21	000000_00101_01000_00101_00000_100000	00A82820	add \$5, \$5, \$8
22	000010_0000000000000000000000000110	08000006	j loop_start (<i>Jump target 6</i>)
— <i>end: (PC = 23)</i> —			
23	001000_00000_00010_00000000000001010	2002000A	addi \$v0, \$0, 10 (<i>\$v0 este \$2</i>)
24	000000_00000_00000_00000_00000_001100	0000000C	syscall (<i>cod syscall generic</i>)

Tabela 1: Maparea detaliată între codul sursă și memoria ROM

Partea I

Implementarea cu Ciclu Unic

Această parte detaliază implementarea inițială a procesorului, în care fiecare instrucțiune se execută complet într-un singur ciclu de ceas.

4 Setul de Instrucțiuni și Semnalele de Control

Procesorul decodifică un subset de 10 instrucțiuni MIPS 32. Următorul tabel prezintă codificările binare și semnalele generate de Unitatea de Control (UC).

Instrucțiune	Opcode [31-26]	RegDst	ExtOp	ALUSrc	Branch	Br_ne	Jump	MemWrite	MemtoReg	RegWrite	ALUOp[1:0]	function [5-0]	ALUCtrl[2:0]
add	000000	1	0	0	0	0	0	0	0	1	10	100000	000 (+)
and	000000	1	0	0	0	0	0	0	0	1	10	100100	010 (&)
slt	000000	1	0	0	0	0	0	0	0	1	10	101010	011 (<)
sll	000000	1	0	0	0	0	0	0	0	1	10	000000	100 (<<)
addi	001000	0	1	1	0	0	0	0	0	1	00 (+)	X	000 (+)
lw	100011	0	1	1	0	0	0	0	1	1	00 (+)	X	000 (+)
sw	101011	0	1	1	0	0	0	1	0	0	00 (+)	X	000 (+)
beq	000100	0	1	0	1	0	0	0	0	0	01 (-)	X	001 (-)
bne	000101	0	1	0	0	1	0	0	0	0	01 (-)	X	001 (-)
j	000010	0	0	0	0	0	1	0	0	0	00	X	000

Tabela 2: Codificările instrucțiunilor și semnalele de control

4.1 Clasificarea Instrucțiunilor și Calea de Date

4.1.1 Instrucțiuni de Tip R (Register)

Folosește trei registre: sursele **rs**, **rt** și destinația **rd**. ALU efectuează operația indicată de **funct**.

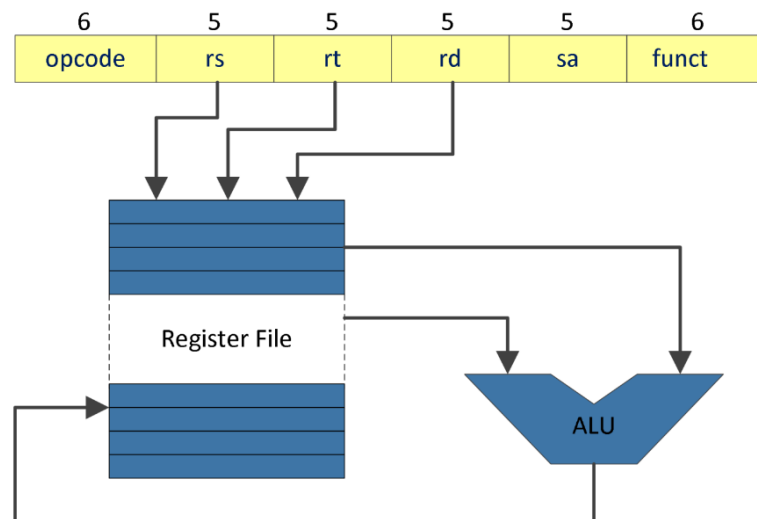


Figura 1: Calea de date pentru instrucțiunile de tip R

4.1.2 Instrucțiuni de Tip I (Immediate)

Folosește o constantă imediată de 16 biți, extinsă la 32 de biți (cu semn sau zero) înainte de a intra în ALU.

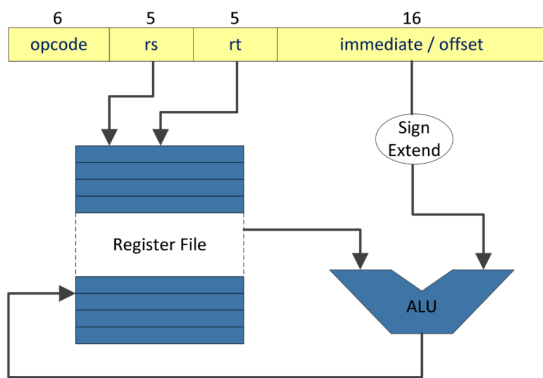


Figura 2: Extindere cu semn (`addi`)

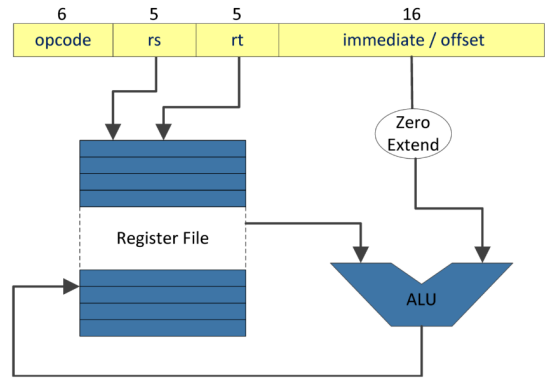


Figura 3: Extindere cu zero

Accesul la Memorie (`lw` și `sw`) Adresa de memorie este suma dintre registrul bază și offset-ul extins cu semn.

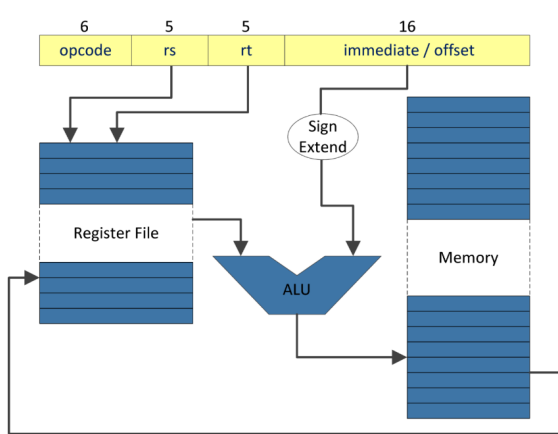


Figura 4: Încărcare (`lw`)

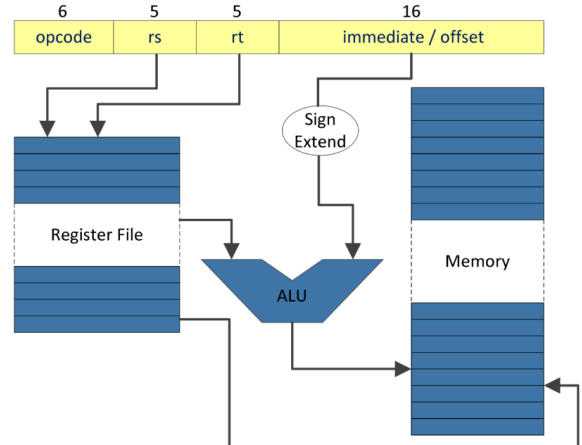


Figura 5: Stocare (`sw`)

Salt Condiționat (`beq`, `bne`) Decizia de salt se bazează pe verificarea egalității operanzilor în ALU.

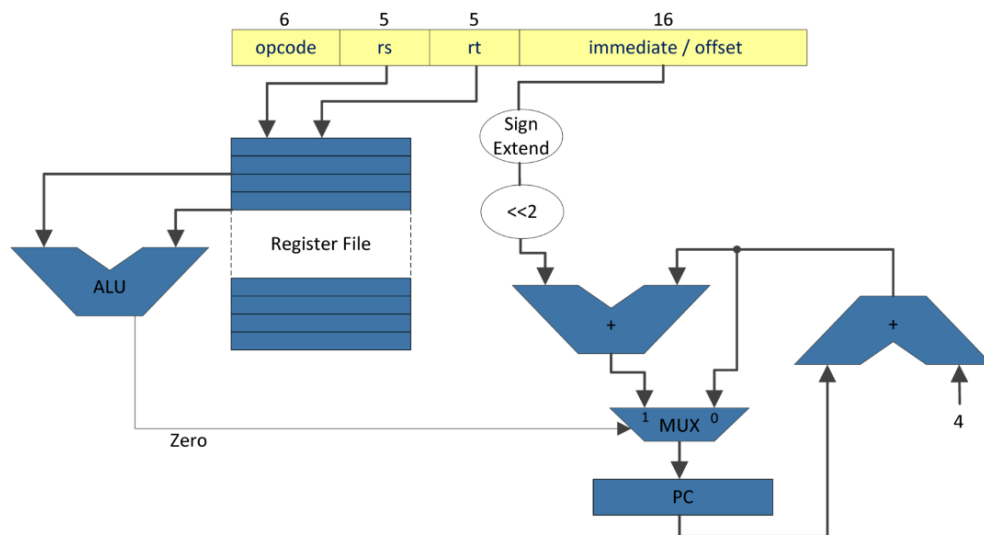


Figura 6: Instrucțiunile de salt condiționat

4.1.3 Instrucțiuni de Tip J (Jump)

Modifică direct Contorul de Program cu adresa furnizată.

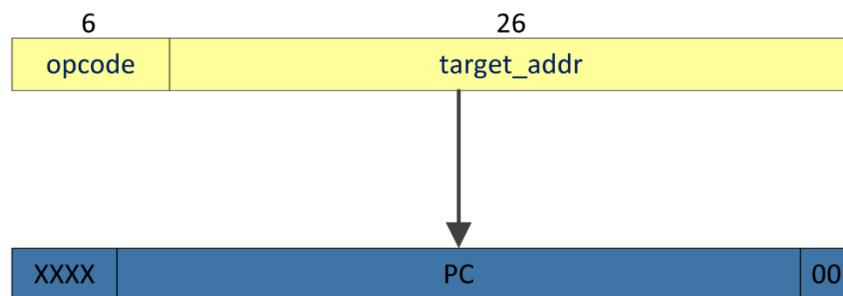


Figura 7: Instrucțiunile de salt necondiționat

5 Componentele Hardware și Arhitectura

5.1 Diagrama Bloc a Procesorului

Diagrama ilustrează interconectarea modulelor IFetch, IDecode, EX și MEM. A fost extinsă logic pentru a suporta instrucțiunea **bne**.

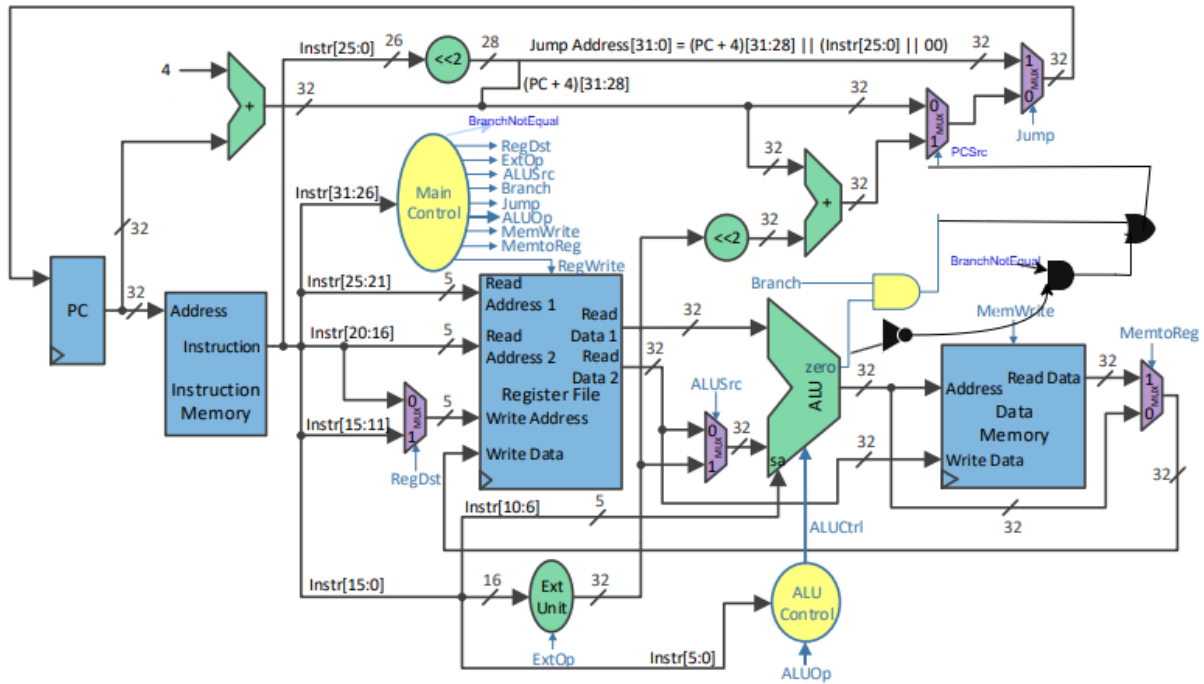


Figura 8: Arhitectura MIPS cu Ciclu Unic

5.2 IFetch.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity IFetch is
6      Port ( clk          : in  STD_LOGIC;
7            rst          : in  STD_LOGIC;
8            en           : in  STD_LOGIC;
9            BranchAddress : in  STD_LOGIC_VECTOR (31 downto 0);
10           JumpAddress  : in  STD_LOGIC_VECTOR (31 downto 0);
11           Jump         : in  STD_LOGIC;
12           PCSrc        : in  STD_LOGIC;
13           Instruction   : out STD_LOGIC_VECTOR (31 downto 0);
14           PC_plus_4     : out STD_LOGIC_VECTOR (31 downto 0));
15 end IFetch;
16
17 architecture Behavioral of IFetch is
18     signal PC          : STD_LOGIC_VECTOR(31 downto 0) := (others => '0');
19     signal Next_PC     : STD_LOGIC_VECTOR(31 downto 0);
20     signal PC4         : STD_LOGIC_VECTOR(31 downto 0);
21     type rom_type is array (0 to 31) of std_logic_vector(31 downto 0);
22     signal ROM : rom_type := (
23         0 => B"000000_00000_00000_00100_00000_100000",
24         1 => B"001000_00000_01010_00000000000010000",
25         2 => B"100011_00100_00001_00000000000000000",
26         3 => B"100011_00100_00010_000000000000000100",
27         4 => B"100011_00100_00011_00000000000001000",
28         5 => B"000000_00000_00000_00101_00000_100000",
29         6 => B"000000_00000_00000_00110_00000_100000",
30         7 => B"000000_00110_00011_00111_00000_101010",
31         8 => B"000100_00111_00000_00000000000001111",

```

```

32     9  => B"000000_00000_00110_01001_00010_000000",
33     10 => B"000000_01010_01001_01001_00000_100000",
34     11 => B"100011_01001_01000_0000000000000000",
35     12 => B"001000_00110_00110_0000000000000001",
36     13 => B"000000_01000_00001_00111_00000_101010",
37     14 => B"000101_00111_00000_1111111111111000",
38     15 => B"000000_00010_01000_00111_00000_101010",
39     16 => B"000101_00111_00000_1111111111110110",
40     17 => B"000000_00000_01000_00111_00000_101010",
41     18 => B"000100_00111_00000_1111111111110100",
42     19 => B"001000_01000_01001_1111111111111111",
43     20 => B"000000_01000_01001_01001_00000_100100",
44     21 => B"000101_01001_00000_1111111111110001",
45     22 => B"000000_00101_01000_00101_00000_100000",
46     23 => B"000010_0000000000000000000000000111",
47     24 => B"101011_00100_00101_0000000000001100",
48     25 => B"000010_000000000000000000000011001",
49     others => B"000000_00000_00000_00000_00000_000000"
50 );
51 begin
52     process(clk, rst)
53     begin
54         if rst = '1' then
55             PC <= (others => '0');
56         elsif rising_edge(clk) then
57             if en = '1' then
58                 PC <= Next_PC;
59             end if;
60         end if;
61     end process;
62
63     PC4 <= PC + 4;
64     PC_plus_4 <= PC4;
65     Next_PC <= JumpAddress when Jump = '1' else
66         BranchAddress when PCSrc = '1' else
67         PC4;
68     Instruction <= ROM(conv_integer(PC(6 downto 2)));
69 end Behavioral;

```

5.3 IDcode.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity IDcode is
6      Port ( clk          : in  STD_LOGIC;
7            en            : in  STD_LOGIC;
8            Instr         : in  STD_LOGIC_VECTOR (25 downto 0);
9            WD            : in  STD_LOGIC_VECTOR (31 downto 0);
10           RegWrite      : in  STD_LOGIC;
11           RegDst        : in  STD_LOGIC;
12           ExtOp         : in  STD_LOGIC;
13           RD1           : out STD_LOGIC_VECTOR (31 downto 0);
14           RD2           : out STD_LOGIC_VECTOR (31 downto 0);
15           Ext_Imm       : out STD_LOGIC_VECTOR (31 downto 0);
16           func          : out STD_LOGIC_VECTOR (5  downto 0);
17           sa            : out STD_LOGIC_VECTOR (4  downto 0));
18 end IDcode;

```

```

19
20 architecture Behavioral of IDecode is
21     type reg_array is array(0 to 31) of STD_LOGIC_VECTOR(31 downto 0);
22     signal reg_file : reg_array := (others => x"00000000");
23     signal WriteAddress : STD_LOGIC_VECTOR(4 downto 0);
24 begin
25     WriteAddress <= Instr(15 downto 11) when RegDst = '1' else Instr(20
        downto 16);
26     process(clk)
27     begin
28         if rising_edge(clk) then
29             if en = '1' and RegWrite = '1' then
30                 reg_file(conv_integer(WriteAddress)) <= WD;
31             end if;
32         end if;
33     end process;
34
35     RD1 <= reg_file(conv_integer(Instr(25 downto 21)));
36     RD2 <= reg_file(conv_integer(Instr(20 downto 16)));
37     Ext_Imm(15 downto 0) <= Instr(15 downto 0);
38     Ext_Imm(31 downto 16) <= (others => Instr(15)) when ExtOp = '1' else (
        others => '0');
39     func <= Instr(5 downto 0);
40     sa <= Instr(10 downto 6);
41 end Behavioral;

```

5.4 EX.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4  use IEEE.NUMERIC_STD.ALL;
5
6  entity EX is
7      Port ( PC_plus_4      : in  STD_LOGIC_VECTOR (31 downto 0);
8            RD1             : in  STD_LOGIC_VECTOR (31 downto 0);
9            RD2             : in  STD_LOGIC_VECTOR (31 downto 0);
10           Ext_Imm          : in  STD_LOGIC_VECTOR (31 downto 0);
11           ALUSrc           : in  STD_LOGIC;
12           ALUOp            : in  STD_LOGIC_VECTOR (1 downto 0);
13           func             : in  STD_LOGIC_VECTOR (5 downto 0);
14           sa               : in  STD_LOGIC_VECTOR (4 downto 0);
15           BranchAddress    : out STD_LOGIC_VECTOR (31 downto 0);
16           ALURes           : out STD_LOGIC_VECTOR (31 downto 0);
17           Zero             : out STD_LOGIC);
18 end EX;
19
20 architecture Behavioral of EX is
21     signal ALUCtrl : STD_LOGIC_VECTOR(2 downto 0);
22     signal A, B    : STD_LOGIC_VECTOR(31 downto 0);
23     signal ALU_Out : STD_LOGIC_VECTOR(31 downto 0);
24 begin
25     BranchAddress <= PC_plus_4 + (Ext_Imm(29 downto 0) & "00");
26     A <= RD1;
27     B <= Ext_Imm when ALUSrc = '1' else RD2;
28
29     process(ALUOp, func)
30     begin
31         case ALUOp is

```

```

32     when "00" => ALUCtrl <= "000";
33     when "01" => ALUCtrl <= "001";
34     when "10" =>
35         case func is
36             when "100000" => ALUCtrl <= "000";
37             when "100100" => ALUCtrl <= "010";
38             when "101010" => ALUCtrl <= "011";
39             when "000000" => ALUCtrl <= "100";
40             when others => ALUCtrl <= "000";
41         end case;
42     when others => ALUCtrl <= "000";
43 end case;
44 end process;
45
46 process(A, B, ALUCtrl, sa)
47 begin
48     case ALUCtrl is
49         when "000" => ALU_Out <= A + B;
50         when "001" => ALU_Out <= A - B;
51         when "010" => ALU_Out <= A and B;
52         when "011" =>
53             if signed(A) < signed(B) then ALU_Out <= x"00000001";
54             else ALU_Out <= x"00000000"; end if;
55         when "100" => ALU_Out <= to_stdlogicvector(to_bitvector(B) sll
56             conv_integer(sa));
57         when others => ALU_Out <= (others => '0');
58     end case;
59 end process;
60
61 ALURes <= ALU_Out;
62 Zero <= '1' when ALU_Out = x"00000000" else '0';
63 end Behavioral;

```

5.5 MEM.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity MEM is
6      Port ( clk          : in  STD_LOGIC;
7            en            : in  STD_LOGIC;
8            MemWrite      : in  STD_LOGIC;
9            ALUResin      : in  STD_LOGIC_VECTOR (31 downto 0);
10           RD2           : in  STD_LOGIC_VECTOR (31 downto 0);
11           MemData       : out STD_LOGIC_VECTOR (31 downto 0);
12           ALUResout     : out STD_LOGIC_VECTOR (31 downto 0));
13 end MEM;
14
15 architecture Behavioral of MEM is
16     type ram_type is array (0 to 63) of STD_LOGIC_VECTOR(31 downto 0);
17     signal RAM : ram_type := (
18         0 => x"00000002", -- RAM[0]: a = 2
19         1 => x"0000005A", -- RAM[4]: b = 90
20         2 => x"00000007", -- RAM[8]: n = 7
21         3 => x"00000000", -- RAM[12]: sum = 0
22         4 => x"00000000", -- RAM[16]: x[0] = 0
23         5 => x"00000001", -- RAM[20]: x[1] = 1
24         6 => x"00000002", -- RAM[24]: x[2] = 2

```

```

25     7 => x"00000003", -- RAM[28]: x[3] = 3
26     8 => x"00000100", -- RAM[32]: x[4] = 256
27     9 => x"0000000E", -- RAM[36]: x[5] = 14
28    10 => x"00000010", -- RAM[40]: x[6] = 16
29     others => x"00000000"
30 );
31
32 begin
33     process(clk)
34     begin
35         if rising_edge(clk) then
36             if en = '1' and MemWrite = '1' then
37                 RAM(conv_integer(ALUResin(7 downto 2))) <= RD2;
38             end if;
39         end if;
40     end process;
41     MemData <= RAM(conv_integer(ALUResin(7 downto 2)));
42     ALUResout <= ALUResin;
43 end Behavioral;

```

5.6 UC.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity UC is
5      Port ( Instr_opcode : in  STD_LOGIC_VECTOR (5 downto 0);
6            RegDst       : out STD_LOGIC;
7            ExtOp        : out STD_LOGIC;
8            ALUSrc       : out STD_LOGIC;
9            Branch       : out STD_LOGIC;
10           BranchNotEq  : out STD_LOGIC;
11           Jump         : out STD_LOGIC;
12           ALUOp        : out STD_LOGIC_VECTOR (1 downto 0);
13           MemWrite     : out STD_LOGIC;
14           MemtoReg     : out STD_LOGIC;
15           RegWrite     : out STD_LOGIC);
16 end UC;
17
18 architecture Behavioral of UC is
19 begin
20     process(Instr_opcode)
21     begin
22         RegDst <= '0'; ExtOp <= '0'; ALUSrc <= '0'; Branch <= '0';
23         BranchNotEq <= '0'; Jump <= '0'; MemWrite <= '0'; MemtoReg <= '0';
24         RegWrite <= '0'; ALUOp <= "00";
25
26         case Instr_opcode is
27             when "000000" => RegDst <= '1'; RegWrite <= '1'; ALUOp <= "10";
28             when "001000" => ExtOp <= '1'; ALUSrc <= '1'; RegWrite <= '1';
29                 ALUOp <= "00";
30             when "100011" => ExtOp <= '1'; ALUSrc <= '1'; MemtoReg <= '1';
31                 RegWrite <= '1'; ALUOp <= "00";
32             when "101011" => ExtOp <= '1'; ALUSrc <= '1'; MemWrite <= '1';
33                 ALUOp <= "00";
34             when "000100" => ExtOp <= '1'; Branch <= '1'; ALUOp <= "01";
35             when "000101" => ExtOp <= '1'; BranchNotEq <= '1'; ALUOp <= "01";
36             ;
37             when "000010" => Jump <= '1';

```

```

34         when others => null;
35     end case;
36 end process;
37 end Behavioral;

```

5.7 test_env.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity test_env is
6      Port ( clk : in STD_LOGIC;
7            btn : in STD_LOGIC_VECTOR (4 downto 0);
8            sw  : in STD_LOGIC_VECTOR (15 downto 0);
9            led  : out STD_LOGIC_VECTOR (15 downto 0);
10           an   : out STD_LOGIC_VECTOR (7 downto 0);
11           cat  : out STD_LOGIC_VECTOR (6 downto 0));
12 end test_env;
13
14 architecture Behavioral of test_env is
15     component IFetch is
16         Port ( clk, rst, en : in STD_LOGIC;
17               BranchAddress, JumpAddress : in STD_LOGIC_VECTOR (31 downto
18                   0);
19               Jump, PCSrc : in STD_LOGIC;
20               Instruction, PC_plus_4 : out STD_LOGIC_VECTOR (31 downto 0));
21     end component;
22
23     component UC is
24         Port ( Instr_opcode : in STD_LOGIC_VECTOR (5 downto 0);
25               RegDst, ExtOp, ALUSrc, Branch, BranchNotEq, Jump : out
26                   STD_LOGIC;
27               ALUOp : out STD_LOGIC_VECTOR (1 downto 0);
28               MemWrite, MemtoReg, RegWrite : out STD_LOGIC);
29     end component;
30
31     component IDecode is
32         Port ( clk, en : in STD_LOGIC;
33               Instr : in STD_LOGIC_VECTOR (25 downto 0);
34               WD : in STD_LOGIC_VECTOR (31 downto 0);
35               RegWrite, RegDst, ExtOp : in STD_LOGIC;
36               RD1, RD2, Ext_Imm : out STD_LOGIC_VECTOR (31 downto 0);
37               func : out STD_LOGIC_VECTOR (5 downto 0);
38               sa : out STD_LOGIC_VECTOR (4 downto 0));
39     end component;
40
41     component EX is
42         Port ( PC_plus_4, RD1, RD2, Ext_Imm : in STD_LOGIC_VECTOR (31 downto
43             0);
44               ALUSrc : in STD_LOGIC;
45               ALUOp : in STD_LOGIC_VECTOR (1 downto 0);
46               func : in STD_LOGIC_VECTOR (5 downto 0);
47               sa : in STD_LOGIC_VECTOR (4 downto 0);
48               BranchAddress, ALURes : out STD_LOGIC_VECTOR (31 downto 0);
49               Zero : out STD_LOGIC);
50     end component;
51
52     component MEM is

```

```

50     Port ( clk, en, MemWrite : in STD_LOGIC;
51           ALUResin, RD2 : in STD_LOGIC_VECTOR (31 downto 0);
52           MemData, ALUResout : out STD_LOGIC_VECTOR (31 downto 0));
53 end component;
54
55 component MPG is
56     Port ( enable : out STD_LOGIC;
57           btn, clk : in STD_LOGIC);
58 end component;
59
60 component SSD is
61     Port ( clk : in STD_LOGIC;
62           digits : in STD_LOGIC_VECTOR(31 downto 0);
63           an : out STD_LOGIC_VECTOR(7 downto 0);
64           cat : out STD_LOGIC_VECTOR(6 downto 0));
65 end component;
66
67 signal en, rst : STD_LOGIC;
68 signal Instruction, PC_plus_4 : STD_LOGIC_VECTOR(31 downto 0);
69 signal RegDst, ExtOp, ALUSrc, Branch, BranchNotEq, Jump, MemWrite,
70       MemtoReg, RegWrite : STD_LOGIC;
71 signal ALUOp : STD_LOGIC_VECTOR(1 downto 0);
72 signal RD1, RD2, Ext_Imm : STD_LOGIC_VECTOR(31 downto 0);
73 signal func : STD_LOGIC_VECTOR(5 downto 0);
74 signal sa : STD_LOGIC_VECTOR(4 downto 0);
75 signal BranchAddress, ALURes, MemData, ALUResout, WD, JumpAddress :
76       STD_LOGIC_VECTOR(31 downto 0);
77 signal Zero, PCSrc : STD_LOGIC;
78 signal digits_out : STD_LOGIC_VECTOR(31 downto 0);
79
80 begin
81     empege: MPG port map (enable => en, btn => btn(0), clk => clk);
82     rst <= btn(1);
83     display: SSD port map (clk => clk, digits => digits_out, an => an, cat
84         => cat);
85
86     inst_IFetch: IFetch port map (clk, rst, en, BranchAddress, JumpAddress,
87         Jump, PCSrc, Instruction, PC_plus_4);
88     inst_UC: UC port map (Instruction(31 downto 26), RegDst, ExtOp, ALUSrc,
89         Branch, BranchNotEq, Jump, ALUOp, MemWrite, MemtoReg, RegWrite);
90     inst_IDeode: IDeode port map (clk, en, Instruction(25 downto 0), WD,
91         RegWrite, RegDst, ExtOp, RD1, RD2, Ext_Imm, func, sa);
92     inst_EX: EX port map (PC_plus_4, RD1, RD2, Ext_Imm, ALUSrc, ALUOp, func,
93         sa, BranchAddress, ALURes, Zero);
94     inst_MEM: MEM port map (clk, en, MemWrite, ALURes, RD2, MemData,
95         ALUResout);
96
97     WD <= MemData when MemtoReg = '1' else ALUResout;
98     PCSrc <= (Branch and Zero) or (BranchNotEq and not Zero);
99     JumpAddress <= PC_plus_4(31 downto 28) & Instruction(25 downto 0) & "00"
100         ;
101
102     led(0) <= RegDst; led(1) <= ExtOp; led(2) <= ALUSrc; led(3) <= Branch;
103     led(4) <= BranchNotEq; led(5) <= Jump; led(6) <= MemWrite; led(7) <=
104         MemtoReg;
105     led(8) <= RegWrite; led(10 downto 9) <= ALUOp; led(15 downto 11) <= (
106         others => '0');
107
108     process(sw(7 downto 5), Instruction, PC_plus_4, RD1, RD2, Ext_Imm,
109         ALURes, MemData, WD)

```

```

98     begin
99         case sw(7 downto 5) is
100             when "000" => digits_out <= Instruction;
101             when "001" => digits_out <= PC_plus_4;
102             when "010" => digits_out <= RD1;
103             when "011" => digits_out <= RD2;
104             when "100" => digits_out <= Ext_Imm;
105             when "101" => digits_out <= ALURes;
106             when "110" => digits_out <= MemData;
107             when "111" => digits_out <= WD;
108             when others => digits_out <= (others => '0');
109         end case;
110     end process;
111 end Behavioral;

```

6 Urmărirea Execuției în Ciclu Unic

În prima iterație, indexul $i = 0$, iar valoarea citită din memorie este $x[0] = 0$.

Pas	SW(7:5)	"000"	"001"	"010"	"011"	"100"	"101"	"110"	"111"	Doar pentru salt	
	Instr (assembly)	Instr (hex)	PC+4	RD1	RD2	Ext_Imm	ALURes	MemData	WD	BranchAddr	JumpAddr
0	addi \$1, \$0, 2	20010002	00000004	00000000	00000000	00000002	00000002	XXXXXXXX	00000002	XXXXXXXX	XXXXXXXX
1	addi \$2, \$0, 90	2002005A	00000008	00000000	00000000	0000005A	0000005A	XXXXXXXX	0000005A	XXXXXXXX	XXXXXXXX
2	addi \$3, \$0, 7	20030007	0000000C	00000000	00000000	00000007	00000007	XXXXXXXX	00000007	XXXXXXXX	XXXXXXXX
3	add \$4, \$0, \$0	00002020	00000010	00000000	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
4	add \$5, \$0, \$0	00002820	00000014	00000000	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
5	add \$6, \$0, \$0	00003020	00000018	00000000	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
6	slt \$7, \$6, \$3	00C3382A	0000001C	00000000	00000007	XXXXXXXX	00000001	XXXXXXXX	00000001	XXXXXXXX	XXXXXXXX
7	beq \$7, \$0, end	10E0000F	00000020	00000001	00000000	0000000F	00000001	XXXXXXXX	XXXXXXXX	0000005C	XXXXXXXX
8	sll \$9, \$6, 2	00064880	00000024	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
9	add \$9, \$4, \$9	00894820	00000028	00000000	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
10	lw \$8, 0(\$9)	8D280000	0000002C	00000000	00000000	00000000	00000000	00000000	00000000	XXXXXXXX	XXXXXXXX
11	addi \$6, \$6, 1	20C60001	00000030	00000000	XXXXXXXX	00000001	00000001	XXXXXXXX	00000001	XXXXXXXX	XXXXXXXX
12	slt \$7, \$8, \$1	0101382A	00000034	00000000	00000002	XXXXXXXX	00000001	XXXXXXXX	00000001	XXXXXXXX	XXXXXXXX
13	bne \$7, \$0, loop	14E0FF8	00000038	00000001	00000000	FFFFFFF8	00000001	XXXXXXXX	XXXXXXXX	00000018	XXXXXXXX
— A doua iterație (reia loop_start) —											
14	slt \$7, \$6, \$3	00C3382A	0000003C	00000001	00000007	XXXXXXXX	00000001	XXXXXXXX	00000001	XXXXXXXX	XXXXXXXX

Tabela 3: Urmărirea execuției (prima iterație completă)

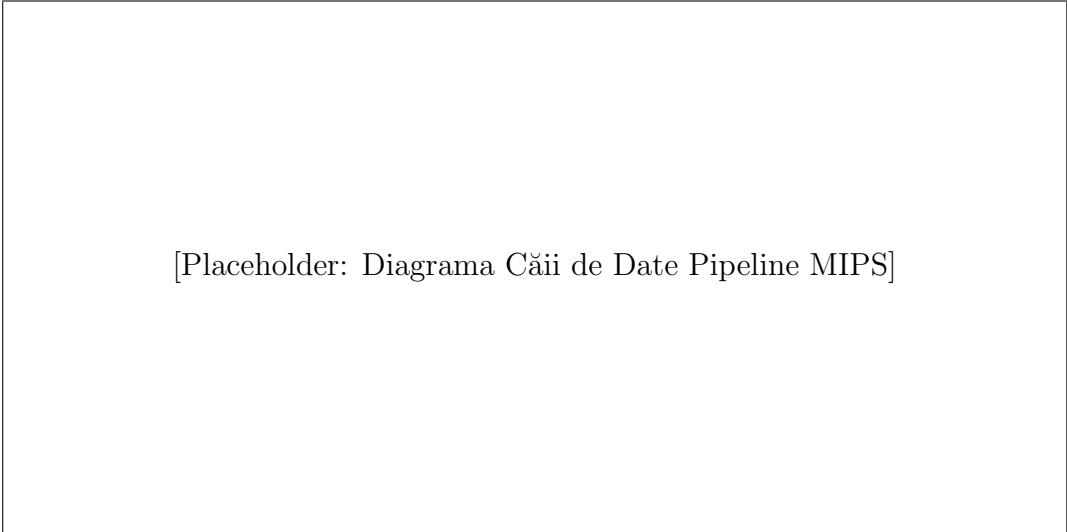
Partea II

Implementarea Pipeline

Această parte detaliază transformarea procesorului într-o arhitectură pipeline cu 5 etape (IF, ID, EX, MEM, WB) pentru a îmbunătăți debitul instrucțiunilor, permițând procesarea simultană a mai multor instrucțiuni.

7 Registrele Pipeline și Arhitectura

Pentru a suporta execuția simultană a instrucțiunilor, au fost introduse registre intermediare între etapele de procesare pentru a propaga datele și semnalele de control sincron. S-a implementat decizia structurală de a rezolva registrul destinație în etapa EX, mutând multiplexorul `RegDst` corespunzător.



[Placeholder: Diagrama Căii de Date Pipeline MIPS]

Figura 9: Arhitectura Pipeline MIPS - Calea de Date

REG_IF_ID [63:0]	REG_ID_EX [157:0]	REG_EX_MEM [106:0]	REG_MEM_WB [70:0]
Instruction [63:32]	MemToReg [157]	MemToReg [106]	MemToReg [70]
PC_plus_4 [31:0]	RegWrite [156]	RegWrite [105]	RegWrite [69]
	MemWrite [155]	MemWrite [104]	MemData [68:37]
	Branch [154]	Branch [103]	ALUResout [36:5]
	BranchNotEq [153]	BranchNotEq [102]	WA [4:0]
	ALUOp [152:151]	BranchAddress [101:70]	
	ALUSrc [150]	Zero [69]	
	RegDst [149]	ALURes [68:37]	
	PC_plus_4 [148:117]	RD2 [36:5]	
	RD1 [116:85]	WA [4:0]	
	RD2 [84:53]		
	Ext_Imm [52:21]		
	func [20:15]		
	sa [14:10]		
	rt [9:5]		
	rd [4:0]		

Tabela 4: Maparea la nivel de biți a Registrelor Pipeline

8 Semnalele de Control în Pipeline

Semnalele de control generate de Unitatea de Control sunt propagate sincron prin registrele pipeline. Tabelul de mai jos detaliază semnalele pentru instrucțiunile suportate, inclusiv semnalul specific **BranchNotEq** folosit pentru instrucțiunea **bne**.

Instrucțiune	Opcode [31:26]	RegDst	ExtOp	ALUSrc	Branch	Br_ne	Jump	MemWrite	MemtoReg	RegWrite	ALUOp[1:0]	function [5:0]	ALUCtrl[2:0]
add	000000	1	0	0	0	0	0	0	0	1	10	100000	000 (+)
and	000000	1	0	0	0	0	0	0	0	1	10	100100	010 (&)
slt	000000	1	0	0	0	0	0	0	0	1	10	101010	011 (<)
sll	000000	1	0	0	0	0	0	0	0	1	10	000000	100 (<<)
addi	001000	0	1	1	0	0	0	0	0	1	00 (+)	X	000 (+)
lw	100011	0	1	1	0	0	0	0	1	1	00 (+)	X	000 (+)
sw	101011	0	1	1	0	0	0	1	0	0	00 (+)	X	000 (+)
beq	000100	X	1	0	1	0	0	0	X	0	01 (-)	X	001 (-)
bne	000101	X	1	0	0	1	0	0	X	0	01 (-)	X	001 (-)
j	000010	X	X	X	0	0	1	0	X	0	XX	X	000

Tabela 5: Codificările instrucțiunilor și maparea semnalelor de control

9 Rezolvarea Hazardurilor

Suprapunerea instrucțiunilor a introdus Hazarduri de Date (Read-After-Write) și Hazarduri de Control (Branch/Jump). Deoarece arhitectura nu implementează o Unitate de Forwarding hardware, hazardurile au fost rezolvate prin software prin inserarea de instrucțiuni **NoOp** (Nicio Operație) în codul mașină.

- **Hazarduri de Date (RAW):** Scrierea în fișierul de registre are loc în etapa WB. Citirea are loc în etapa ID. Sunt necesare până la 2 instrucțiuni **NoOp** pentru a alinia etapa ID a unei instrucțiuni dependente cu etapa WB a producătorului.
- **Hazarduri de Control (Branch):** Decizia pentru **beq** și **bne** este evaluată în etapa MEM. Prin urmare, 3 instrucțiuni **NoOp** (Sloturi de Întârziere) sunt inserate după fiecare salt condiționat pentru a preveni preluarea unor instrucțiuni incorecte.
- **Hazarduri de Control (Jump):** Adresa țintă este calculată în etapa ID, necesitând doar 1 instrucțiune **NoOp**.

10 Urmărirea Execuției în Pipeline

10.1 Execuție Ideală (Identificarea Hazardurilor)

Tabelul de mai jos ilustrează execuția teoretică a primei iterații a buclei, presupunând că nu există întârzieri. Hazardurile Structurale (S), de Date (D) și de Control (C) sunt marcate explicit în etapele în care apar, specificând instrucțiunile dependente.

Poz.	Instrucțiune	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16	C17	C18	C19	C20	C21	C22	C23	C24	C25	C26	C27
0	addi \$1, \$0, 2	IF	ID	EX	MEM	WB																						
1	addi \$2, \$0, 90		IF	ID	EX	MEM	WB																					
2	addi \$3, \$0, 7			IF	ID	EX	MEM	WB																				
3	add \$4, \$0, \$0				IF	ID	EX	MEM	WB																			
4	add \$5, \$0, \$0					IF	ID	EX	MEM	WB																		
5	add \$6, \$0, \$0						IF	ID	EX	MEM	WB(\$6-D:6)																	
6	slt \$7, \$6, \$3							IF	ID(\$6)	EX	MEM	WB(\$7-D:7)																
7	beq \$7, \$0, end								IF	ID(\$7)	EX	MEM(C)	WB															
8	add \$9, \$4, \$6									IF	ID	EX	MEM	WB(\$9-D:9)														
9	lw \$8, 0(\$9)										IF	ID(\$9)	EX	MEM	WB(\$8-D:11)													
10	addi \$6, \$6, 1											IF	ID	EX	MEM	WB												
11	slt \$7, \$8, \$1												IF	ID(\$8)	EX	MEM	WB(\$7-D:12..14)											
12	bne \$7, \$0, loop													IF	ID(\$7)	EX	MEM(C)	WB										
13	slt \$7, \$2, \$8														IF	ID	EX	MEM	WB(\$7-D:14..16)									
14	bne \$7, \$0, loop															IF	ID(\$7)	EX	MEM(C)	WB								
15	slt \$7, \$0, \$8																IF	ID	EX	MEM	WB(\$7-D:16)							
16	beq \$7, \$0, loop																	IF	ID(\$7)	EX	MEM(C)	WB						
17	addi \$9, \$8, -1																		IF	ID	EX	MEM	WB(\$9-D:18)					
18	and \$9, \$8, \$9																			IF	ID(\$9)	EX	MEM	WB(\$9-D:19)				
19	bne \$9, \$0, loop																				IF	ID(\$9)	EX	MEM(C)	WB			
20	add \$5, \$5, \$8																					IF	ID	EX	MEM	WB		
21	j loop_start																						IF	ID(C)	EX	MEM	WB	
22	j end																							IF	ID(C)	EX	MEM	WB

Tabela 6: Execuția ideală evidențiind hazardurile structurale, de date și de control

[illegible]

Tabela 7: Execuția pipeline rezolvată care arată cele 61 de cicluri de ceas necesare pentru prima iterație completă a buclei

11 Modificările Codului VHDL (Pipeline)

Calea de date a fost adaptată pentru a transporta semnalele prin registrele pipeline intermediare. Interfețele porturilor pentru modulele IDcode și EX au fost actualizate pentru a reflecta relocarea fizică a logicii de calcul al adresei de scriere.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity test_env_pipeline is
6      Port ( clk : in  STD_LOGIC;
7            btn : in  STD_LOGIC_VECTOR (4 downto 0);
8            sw  : in  STD_LOGIC_VECTOR (15 downto 0);
9            led : out STD_LOGIC_VECTOR (15 downto 0);
10           an  : out STD_LOGIC_VECTOR (7  downto 0);
11           cat : out STD_LOGIC_VECTOR (6  downto 0));
12 end test_env_pipeline;
13
14 architecture Behavioral of test_env_pipeline is
15     -- Declara iile componentelor (IFetch, UC, IDcode, EX, MEM, MPG, SSD)
16     -- sunt omise pentru concizie
17
18     -- Semnale inter-etaje
19     signal en, rst : STD_LOGIC;
20     signal Instruction, PC_plus_4 : STD_LOGIC_VECTOR(31 downto 0);
21     signal PCSrc, Jump : STD_LOGIC;
22     signal JumpAddress : STD_LOGIC_VECTOR(31 downto 0);
23     signal RegDst, ExtOp, ALUSrc, Branch, BranchNotEq, MemWrite, MemtoReg,
24         RegWrite : STD_LOGIC;
25     signal ALUOp : STD_LOGIC_VECTOR(1 downto 0);
26     signal RD1, RD2, Ext_Imm : STD_LOGIC_VECTOR(31 downto 0);
27     signal func : STD_LOGIC_VECTOR(5 downto 0);
28     signal sa : STD_LOGIC_VECTOR(4 downto 0);
29     signal BranchAddress, ALURes, MemData, ALUResout : STD_LOGIC_VECTOR(31
30         downto 0);
31     signal Zero : STD_LOGIC;
32     signal WA_out_EX : STD_LOGIC_VECTOR(4 downto 0);
33     signal WD : STD_LOGIC_VECTOR(31 downto 0);
34
35     -- REGISTRE PIPELINE
36     -- 1. IF/ID
37     signal IF_ID_PC_plus_4, IF_ID_Instr : STD_LOGIC_VECTOR(31 downto 0);
38
39     -- 2. ID/EX
40     signal ID_EX_PC_plus_4, ID_EX_RD1, ID_EX_RD2, ID_EX_Ext_Imm :
41         STD_LOGIC_VECTOR(31 downto 0);
42     signal ID_EX_func : STD_LOGIC_VECTOR(5 downto 0);
43     signal ID_EX_sa, ID_EX_rt, ID_EX_rd : STD_LOGIC_VECTOR(4 downto 0);
44     signal ID_EX_RegDst, ID_EX_ALUSrc, ID_EX_Branch, ID_EX_BranchNotEq,
45         ID_EX_MemWrite, ID_EX_MemtoReg, ID_EX_RegWrite : STD_LOGIC;
46     signal ID_EX_ALUOp : STD_LOGIC_VECTOR(1 downto 0);
47
48     -- 3. EX/MEM
49     signal EX_MEM_BranchAddress, EX_MEM_ALURes, EX_MEM_RD2 :
50         STD_LOGIC_VECTOR(31 downto 0);
51     signal EX_MEM_Zero, EX_MEM_Branch, EX_MEM_BranchNotEq, EX_MEM_MemWrite,
52         EX_MEM_MemtoReg, EX_MEM_RegWrite : STD_LOGIC;
53     signal EX_MEM_WA : STD_LOGIC_VECTOR(4 downto 0);
```

```

48  -- 4. MEM/WB
49  signal MEM_WB_MemData, MEM_WB_ALURes : STD_LOGIC_VECTOR(31 downto 0);
50  signal MEM_WB_WA : STD_LOGIC_VECTOR(4 downto 0);
51  signal MEM_WB_MemtoReg, MEM_WB_RegWrite : STD_LOGIC;
52
53  begin
54  -- Instanrierile MPG, SSD i ale modulelor mapate la semnalele
    pipeline...
55
56  process(clk)
57  begin
58      if rising_edge(clk) then
59          if en = '1' then
60              -- IF -> ID
61              IF_ID_PC_plus_4 <= PC_plus_4;
62              IF_ID_Instr <= Instruction;
63
64              -- ID -> EX
65              ID_EX_PC_plus_4 <= IF_ID_PC_plus_4;
66              ID_EX_RD1 <= RD1;
67              ID_EX_RD2 <= RD2;
68              ID_EX_Ext_Imm <= Ext_Imm;
69              ID_EX_func <= func;
70              ID_EX_sa <= sa;
71              ID_EX_rt <= IF_ID_Instr(20 downto 16);
72              ID_EX_rd <= IF_ID_Instr(15 downto 11);
73
74              ID_EX_RegDst <= RegDst;
75              ID_EX_ALUSrc <= ALUSrc;
76              ID_EX_ALUOp <= ALUOp;
77              ID_EX_Branch <= Branch;
78              ID_EX_BranchNotEq <= BranchNotEq;
79              ID_EX_MemWrite <= MemWrite;
80              ID_EX_MemtoReg <= MemtoReg;
81              ID_EX_RegWrite <= RegWrite;
82
83              -- EX -> MEM
84              EX_MEM_BranchAddress <= BranchAddress;
85              EX_MEM_ALURes <= ALURes;
86              EX_MEM_RD2 <= ID_EX_RD2;
87              EX_MEM_Zero <= Zero;
88              EX_MEM_WA <= WA_out_EX;
89
90              EX_MEM_Branch <= ID_EX_Branch;
91              EX_MEM_BranchNotEq <= ID_EX_BranchNotEq;
92              EX_MEM_MemWrite <= ID_EX_MemWrite;
93              EX_MEM_MemtoReg <= ID_EX_MemtoReg;
94              EX_MEM_RegWrite <= ID_EX_RegWrite;
95
96              -- MEM -> WB
97              MEM_WB_MemData <= MemData;
98              MEM_WB_ALURes <= ALUResout;
99              MEM_WB_WA <= EX_MEM_WA;
100
101              MEM_WB_MemtoReg <= EX_MEM_MemtoReg;
102              MEM_WB_RegWrite <= EX_MEM_RegWrite;
103          end if;
104      end if;
105  end process;
106

```

```

107     WD <= MEM_WB_MemData when MEM_WB_MemtoReg = '1' else MEM_WB_ALURes;
108     PCSrc <= (EX_MEM_Branch and EX_MEM_Zero) or (EX_MEM_BranchNotEq and not
109             EX_MEM_Zero);
110     JumpAddress <= IF_ID_PC_plus_4(31 downto 28) & IF_ID_Instr(25 downto 0)
111             & "00";
112 end Behavioral;

```

test_env_pipeline.vhd (Top-Level Pipeline)

Partea III

Concluzii

12 Sinteza FPGA

Ambele arhitecturi (Ciclu Unic și Pipeline) au fost sintetizate cu succes în mediul Vivado. Fișierele `.bit` rezultate au fost programate pe placa FPGA. Execuția s-a dovedit corectă din punct de vedere logic, validând corect algoritmi condiționali, executând salturile condiționate (`beq`, `bne`), logica aritmetică și acumulând corect suma elementelor valide, rezolvând în același timp hazardurile inerente de control și date prin inserarea corectă de NOP-uri în codul mașină.